Chapitre -1 : Introduction

Programmation C Avancée - L3

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

1 Introduction

Ce cours couvre la programmation C avancée pour les étudiants de L3. L'objectif est la compréhension approfondie des mécanismes sous-jacents du langage C, de l'organisation mémoire, et des techniques de programmation système.

2 Pourquoi le C

Le langage C reste pertinent pour plusieurs domaines critiques:

Systèmes d'exploitation

Le noyau Linux (30+ millions de lignes), Windows NT kernel, macOS/iOS kernel (XNU), et les drivers matériels sont écrits en C. La compréhension du C permet de comprendre les interfaces système (POSIX) et le fonctionnement des OS.

Systèmes embarqués

Firmware des microcontrôleurs (ARM Cortex, AVR, PIC), systèmes temps réel (FreeRTOS, Zephyr), dispositifs IoT, équipements médicaux, et systèmes automobiles (CAN bus, ECU) utilisent le C pour son efficacité et son contrôle direct du matériel.

Performance critique

Moteurs de bases de données (PostgreSQL, MySQL, SQLite), runtimes de langages (CPython interpreter, JVM Hotspot), serveurs web (nginx, Apache), bibliothèques numériques (BLAS, LAPACK, FFTW), et codecs multimédia (FFmpeg, x264) sont implémentés en C.

Infrastructure logicielle

Compilateurs (GCC, Clang/LLVM), interpréteurs (Lua, Ruby MRI), shells (bash, zsh), utilitaires système (coreutils), et protocoles réseau (OpenSSH, OpenSSL, curl) sont écrits en C.

Backends AI/ML

Les couches de calcul de TensorFlow, PyTorch (ATen/C10), ONNX Runtime, cuDNN (NVIDIA), et OpenBLAS utilisent du C/C++ pour les opérations matricielles et le calcul GPU.

Sécurité et cryptographie

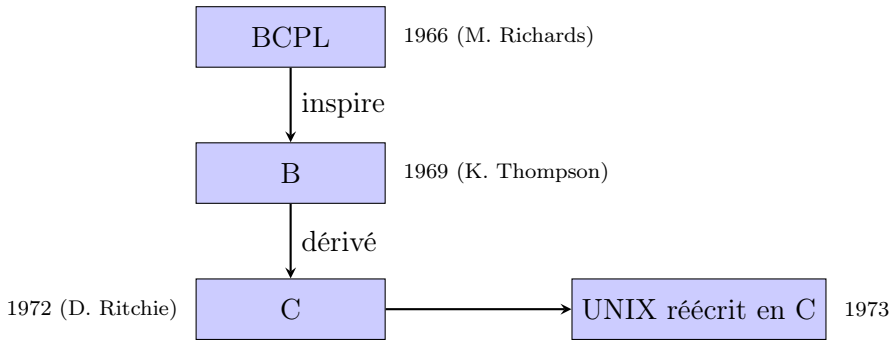
OpenSSL, libsodium, WireGuard, et les implémentations de TLS/SSL sont en C pour minimiser la surface d'attaque et garantir la performance.

Caractéristiques du C

Le C offre un contrôle précis de la mémoire, une prédictibilité des performances, et une portabilité maximale. Ce cours forme des ingénieurs capables de comprendre et maintenir du code système, d'optimiser les performances, et de débugger efficacement.

3 Histoire du C

Le langage C a été développé entre 1969 et 1973 par Dennis Ritchie aux Bell Labs (AT&T). L'objectif initial était de réécrire le système d'exploitation UNIX, précédemment écrit en assembleur.



Le C est directement dérivé du langage B (Ken Thompson, 1969), lui-même inspiré de BCPL (Basic Combined Programming Language, Martin Richards, 1966).

3.1 Versions Majeures

K&R C (1978)

Première définition du langage dans "The C Programming Language" par Brian Kernighan et Dennis Ritchie. Standard de facto sans normalisation formelle.

ANSI C / C89 (1989)

Première standardisation par l'ANSI (American National Standards Institute). Ajout de prototypes de fonctions, qualificateurs `const` et `volatile`, bibliothèque standard.

C99 (1999)

Types `long long`, `_Bool`, nombres complexes. Commentaires `//`. Déclarations mixtes code/variables. Tableaux de longueur variable (VLA). Fonctions `inline`.

C11 (2011)

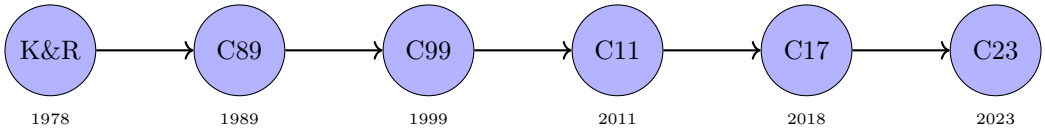
Support multithreading natif (`<threads.h>`), assertions statiques (`_Static_assert`), types atomiques (`<stdatomic.h>`), amélioration Unicode.

C17 (2018)

Corrections techniques, pas de nouvelles fonctionnalités majeures.

C23 (2023)

Dernière version. Ajout de `typeof`, attributs standards (`[[nodiscard]]`), `_BitInt(N)` pour entiers de largeur arbitraire, amélioration des préprocesseur.



3.2 Influence et Philosophie

Influence du C

Le C a directement influencé C++, Objective-C, C#, Java, JavaScript, Go, et Rust. La syntaxe du C (accolades, opérateurs, structures de contrôle) est devenue le standard de facto pour les langages impératifs.

Philosophie de design

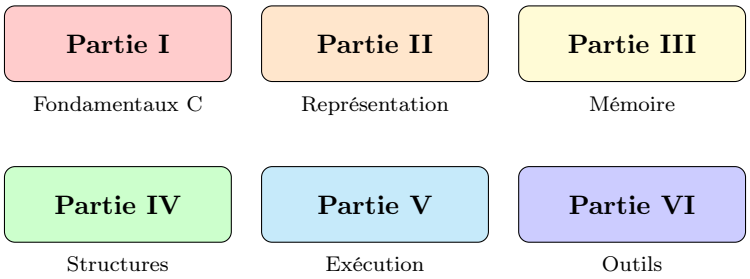
Principes fondamentaux:

- Confiance au programmeur
- Pas de vérifications inutiles au runtime
- Correspondance proche avec le matériel
- Langage minimal avec bibliothèque standard riche

Ces choix expliquent la performance du C mais aussi ses pièges (UB, gestion manuelle de la mémoire).

4 Structure du Cours

Le cours est organisé en 6 grandes parties couvrant 13 chapitres obligatoires:



4.1 Détail des Parties

Partie I: Fondamentaux C

Maîtrise complète de la syntaxe C (structures de contrôle, types, opérateurs, fonctions, compilation).

Partie II: Représentation

Représentation numérique (binaire, hexadécimal, virgule flottante IEEE 754).

Partie III: Mémoire, Pointeurs, Tableaux

Organisation mémoire, adressage, pointeurs, tableaux 1D/2D, chaînes de caractères.

Partie IV: Structures et Allocation

Structures, unions, alignement, allocation dynamique (malloc/free).

Partie V: Exécution et Système

Stack frames, layout mémoire, hiérarchie cache, fichiers POSIX.

Partie VI: Outils et Techniques

Makefiles, linking, pointeurs de fonctions, préprocesseur, optimisation.

Chapitres optionnels

Deux chapitres optionnels couvrent le multithreading et les sockets réseau.

4.2 Objectifs du Cours

Compétences visées

- Comprendre les mécanismes sous-jacents du C (pas seulement la syntaxe)
- Maîtriser l'organisation mémoire et l'adressage
- Debugger efficacement (GDB, Valgrind)
- Optimiser les performances (cache, algorithmes)
- Développer du code système robuste
- Utiliser les outils professionnels (Make, Docker)

5 Prérequis

Connaissances requises

Les étudiants doivent avoir des connaissances basiques en C:

- Variables et types de base (int, float, char)
- Structures de contrôle simples (if, for, while)
- Fonctions et passage de paramètres
- Pointeurs simples (déclaration, déréférencement)
- Compilation basique avec gcc

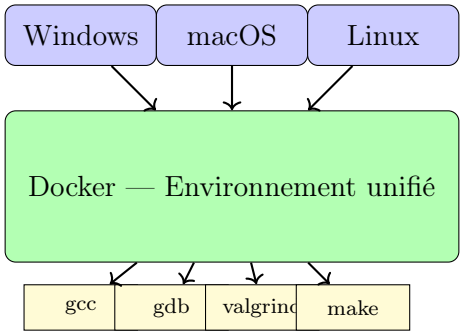
Ce cours n'est pas une introduction au C. Les concepts de base sont supposés acquis.

6 Environnement de Travail

Tous les travaux pratiques (TP) utilisent un environnement Docker standardisé. Le TP-1 couvre l'installation et la configuration de cet environnement.

Avantages de Docker

L'utilisation de Docker garantit un environnement identique pour tous les étudiants, indépendamment du système d'exploitation (Windows, macOS, Linux).



Contenu de l'image Docker

L'image Docker (`elbachirtoure/c-advanced-course:v2.0`) contient:

- **Compilateurs:** gcc, g++
- **Debuggers:** gdb (avec gdb-dashboard pour meilleure visualisation)
- **Outils mémoire:** valgrind, AddressSanitizer
- **Éditeurs:** vim, nano
- **Outils de build:** make, cmake

7 Organisation Pédagogique

Pour chaque chapitre, le cours comprend trois composantes:

CM
Cours Magistral

TD
Travaux Dirigés

TP
Travaux Pratiques

CM (Cours Magistral)

Présentation des concepts avec visualisations mémoire et exemples techniques.

TD (Travaux Dirigés)

Exercices d'analyse et de compréhension approfondie des mécanismes. Inclut des exercices forensics d'analyse de dumps hexadécimaux.

TP (Travaux Pratiques)

Mise en pratique avec compilation, debugging, et implémentation de programmes. Utilisation intensive de GDB et Valgrind.

8 Conclusion

Ce cours vise à former des ingénieurs capables de:

- Comprendre en profondeur le fonctionnement du C
- Analyser et optimiser du code système
- Debugger efficacement des problèmes complexes
- Développer du code robuste et performant